SYSTEM SPECIFICATION & USER MANUAL

# ChatBot with PDF

Retrieval-Augmented document question-answering system — technical specification and end-user guide.

| | | | |
|---|---|---|---|
| Prepared by | **Julius Godwin Kanyama** | Document status | **v1.0 — Final** |
| Date | **07 September 2026** | Classification | **Academic / Portfolio** |
| Project origin | **ISE492 Graduation Project** | Deployment status | **Local / Non-Production** |

## 1 Introduction & Purpose

ChatBot with PDF is a document question-answering system that allows a user to upload a document and receive natural-language answers to questions about its contents, generated using Retrieval-Augmented Generation (RAG) over the document's own text rather than a general-purpose model response. Answers are returned together with a citation to the specific page or slide the information was drawn from.

The system was developed as an academic project (ISE492 graduation project) and is presented here as a supporting technical artifact demonstrating applied work in natural-language processing, information retrieval, and full-stack system design. It is intended for a single end user — a student, researcher, or professional — who needs to query a document's contents without reading it in full, and is not, in its current form, a hardened multi-tenant production service.

## 2 Scope

### IN SCOPE

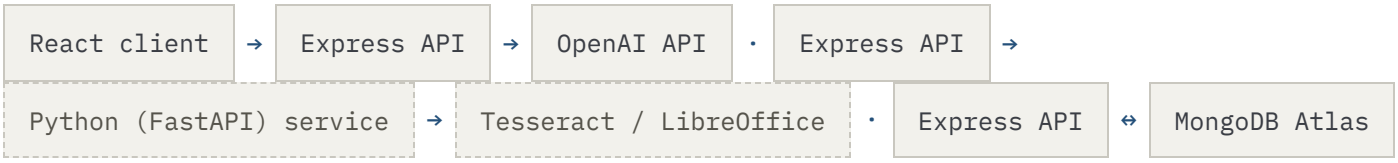
- Upload and processing of PDF, DOCX, and PPTX documents, and of common image formats (JPG, PNG, GIF, BMP, TIFF) via OCR.
- Per-user account registration and authentication.
- Natural-language Q&A over a single document, or jointly over multiple documents grouped into a folder.
- Page/slide-level source citation, with automatic detection of unsupported citations.
- A web interface for uploading, previewing, and chatting with a document.
- Persistence of conversation history per document and per folder.

### OUT OF SCOPE / CURRENT LIMITATIONS

- ☐ Production-grade infrastructure: no CI/CD, containerised deployment, or load testing has been performed.
- ☐ The document-processing microservice is optional and runs as a separate local process; if it is not running, the system falls back automatically to a simpler Node.js-only pipeline.
- ☐ Upload/embedding progress is tracked in memory on the server process and does not survive a restart.
- ☐ PPTX-to-PDF preview conversion depends on LibreOffice being installed on the host machine.
- ☐ OCR is applied automatically to image-file uploads; a scanned (image-only) PDF is detected but not yet routed through OCR automatically during standard upload.
- ☐ No administrative or moderation interface is implemented.
- ☐ The response model is fixed to OpenAI GPT-3.5-turbo and is not user-configurable.

## 3 System Architecture Overview

The system is a three-tier web application. A React single-page application communicates with an Express.js REST API over HTTP/JSON, authenticated by a JWT stored in an httpOnly cookie. The backend follows a Route → Controller → Service/Utility → Model layering, backed by MongoDB Atlas. *(The project also contains legacy EJS templates from an earlier iteration; these now serve only a static translation-file endpoint and are not part of the active user interface.)*




The backend calls the OpenAI API directly for two purposes: generating vector embeddings at document-ingestion time, and generating the final answer at query time. It separately calls an optional Python (FastAPI) microservice, over local HTTP, for higher-quality document text extraction, OCR, and NLP-based chunking; the backend performs a health check before each use and transparently falls back to a built-in Node.js pipeline if that service is unreachable — the Python service is an accelerator, not a hard dependency.

### WHERE OCR FITS

OCR is performed by Tesseract in one of two places: inside the Python microservice (via the `pytesseract` binding) when that service is available and the upload is an image file, or inside the Node.js fallback path (via `tesseract.js`) when the microservice is not running. In both cases the OCR output is plain text that is fed into the same downstream chunking and embedding pipeline used for text extracted directly from PDF, DOCX, or PPTX files.

### TECHNOLOGY SUMMARY

| LAYER | TECHNOLOGY |
|---|---|
| Frontend | `React, Vite, Tailwind CSS` |
| Backend API | `Node.js, Express.js` |
| Authentication | `JWT (httpOnly cookie), bcrypt` |
| Database | `MongoDB Atlas + Atlas Vector Search` |
| Language model | `OpenAI GPT-3.5-turbo` |

| | |
|---|---|
| Embedding model | `OpenAI text-embedding-3-small` |
| Document extraction (optional) | `Python, FastAPI, pdfplumber, python-docx, python-pptx` |
| OCR | `Tesseract (pytesseract / tesseract.js)` |
| PPTX preview conversion | `LibreOffice (headless)` |

## 4 Functional Requirements

**FR-01** The system shall allow a user to register an account and authenticate using an email address and password.

**FR-02** The system shall restrict access to uploaded documents and conversations to the account that owns them.

**FR-03** The system shall allow an authenticated user to upload a document in PDF, DOCX, PPTX, or a common image format (JPG, PNG, GIF, BMP, TIFF).

**FR-04** The system shall extract the textual content of an uploaded document, applying OCR when the input is an image file.

**FR-05** The system shall divide extracted text into overlapping chunks, using a chunk size and overlap specific to the document type, prior to embedding.

**FR-06** The system shall generate a vector embedding for each chunk using the OpenAI text-embedding-3-small model and store it, with its source page/slide number, in MongoDB Atlas.

**FR-07** The system shall detect tabular content, parse it into a structured representation, and store it as a distinct, separately retrievable chunk type.

**FR-08** The system shall extract images embedded in PDF documents and generate a descriptive caption for each, stored as a distinct, retrievable chunk type.

**FR-09** The system shall accept a natural-language question and retrieve the chunks most relevant to it using a combination of vector similarity search and keyword search.

**FR-10** The system shall construct a context-augmented prompt from the retrieved chunks and submit it, with the user's question, to the OpenAI Chat Completions API to generate an answer.

**FR-11** The system shall return each answer with a citation to its source page(s) or slide(s), and shall verify that every citation corresponds to a chunk actually retrieved for that query.

**FR-12** The system shall allow a question to span multiple documents grouped into one folder, attributing each cited fact to its source document.

**FR-13** The system shall report upload and embedding-generation progress to the client while a document is being processed.

**FR-14** The system shall persist conversation and message history per document or folder so a user may resume a prior session.

**FR-15** The system shall present an in-browser preview of the uploaded document, appropriate to its format, alongside the chat interface.

## 5 Data & Retrieval Model

### CHUNKING

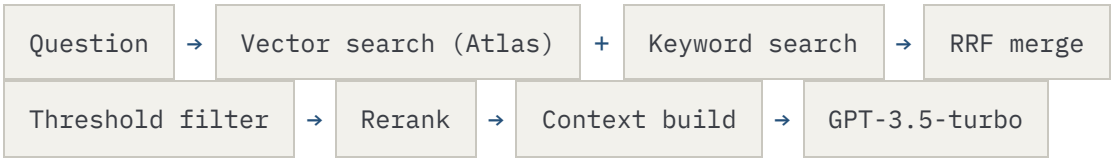
Extracted text is first divided at page boundaries (per-page for PDF; per-slide for PPTX, using slide markers; ~500-word logical sections for DOCX, which has no native page concept). Within each page/slide, text is split into overlapping chunks using a recursive, separator-priority splitter, with chunk size and overlap set per document type: **PDF — 800 tokens / 100 overlap**, **DOCX — 700 / 80**, **PPTX — 500 / 50** (tokens approximated at ~4 characters each). Where the Python microservice is available, chunking instead uses a document-type-specific NLP strategy (spaCy/NLTK, e.g. heading- and section-aware splitting for PDF); the Node.js splitter above is the automatic fallback.

### EMBEDDING

Each chunk is embedded with OpenAI's **text-embedding-3-small** model (1536-dimensional vectors), generated in batches. Every stored embedding record carries its source page/slide number, a chunk type (`text`, `table`, or `image`), and character-offset metadata used for highlighting.

### STORAGE & RETRIEVAL

Embeddings are stored as documents in a MongoDB Atlas collection. Retrieval at query time is **hybrid**: a native MongoDB Atlas `$vectorSearch` query (cosine similarity over the Atlas vector index) runs alongside a MongoDB text-index keyword search, and the two rankings are merged by Reciprocal Rank Fusion. If Atlas Vector Search is not available, the system falls back to computing cosine similarity in application code over the document's stored embeddings. Candidates are then filtered by a minimum similarity threshold (lower for table/image chunks and for page-specific queries), deduplicated, and optionally reranked before a bounded set of chunks (up to 4 text, 2 table, 2 image) is assembled into the context sent to the language model.



## 6 Core User Workflow

1. **Sign in.** The user registers or logs in; a JWT session cookie is issued.
2. **Upload.** The user uploads a PDF, DOCX, PPTX, or image file from the workspace.
3. **Processing.** Text is extracted (with OCR if the file is an image), then chunked and embedded in the background; a progress indicator is shown while this runs.
4. **Preview.** The document is rendered in an in-browser viewer appropriate to its format as soon as it is available.
5. **Ask.** The user types a natural-language question in the chat panel.
6. **Retrieve & answer.** The system retrieves the most relevant chunks, builds a context-augmented prompt, and returns an answer with page/slide citations.
7. **Verify.** The user may click a citation to jump directly to the corresponding page or slide in the viewer.
8. **Resume.** The conversation is saved; the user can return later and continue from where they left off.

## 7 Testing & Quality Assurance Notes

The system was validated through manual, developer-led testing during development. This consisted of uploading representative PDF, DOCX, PPTX, and image documents; asking a range of natural-language questions against them, including page-specific questions and questions with no answer present in the document; and manually confirming that responses were accurate, correctly cited, and that the citation-verification step correctly flagged citations that did not match the retrieved context. No automated test suite, load testing, or formal QA process was applied — testing was informal and exploratory, consistent with the project's scope as an academic and portfolio system rather than a production deployment.

## 8 User Manual

1. **Create an account or sign in.** Open the application and register with an email address and password, or log in if you already have an account.
2. **Upload a document.** From your workspace, choose Upload and select a PDF, Word, or PowerPoint file, or an image (photo of a page, scanned form, etc.).
3. **Wait for processing.** A progress indicator shows while the document's text is being read and prepared for search. Larger documents take longer.
4. **Ask your question.** Once processing is complete, type your question in plain language in the chat panel — for example, "What does section 4 say about eligibility?" or "Summarise page 12."
5. **Read the answer.** The reply appears with a reference to the page or slide it came from, shown in brackets, e.g. `[Page 12]`.
6. **Check the source.** Click a page/slide citation to jump straight to that location in the document preview alongside the chat.
7. **Ask follow-up questions.** The conversation keeps its history, so you can ask follow-up questions without repeating context.
8. **Multiple documents.** To ask a question across several related documents at once, group them into a folder first, then ask your question from the folder view.
9. **Come back later.** Your documents and conversations remain in your account; open the workspace again to continue where you left off.

Note: processing speed and OCR quality depend on document size and image clarity. If the optional document-processing service is offline, uploads still work, using a simpler built-in text extraction method.